

FMAA Ecosystem

Federated Micro-Agents Architecture - Implementation Guide & Results

Implementation Complete

Ready to Deploy

Hasil Implementasi

Komponen Berhasil Diimplementasi

- Database Schema (PostgreSQL/Supabase)
- Agent Factory System
- Sentiment Analysis Agent
- Recommendation Agent
- Performance Monitor Agent
- Flutter Mobile Dashboard

Stack Teknologi

- Vercel - Serverless Functions
- Supabase - Database & Real-time
- Hugging Face - AI Models
- GitHub - Repository & CI/CD
- Flutter - Mobile App
- Node.js - Backend API

Struktur Project

```
fmaa-ecosystem/
├── api/
│   ├── agent-factory.js      # Vercel API Routes
│   ├── sentiment-agent.js   # Core Agent Factory
│   ├── recommendation-agent.js # Sentiment Analysis Agent
│   └── performance-monitor.js # Recommendation Agent
├── database/
│   └── schema.sql           # Performance Monitor
├── mobile-app/
│   ├── lib/main.dart        # Complete Database Schema
│   ├── pubspec.yaml        # Flutter Mobile App
│   └── lib/screens/         # Main App Entry
├── shared/                  # Dependencies
│   ├── utils.js            # Dashboard Screens
│   └── docs/               # Shared Utilities
├── vercel.json              # Common Functions
├── package.json             # Documentation
└── README.md                # Vercel Configuration
                                # Node.js Dependencies
                                # Project Documentation
```

Database Schema

Core Tables

- tenants - Multi-tenancy support
- users - User management
- agents - Agent registry
- agent_tasks - Task management
- agent_metrics - Performance data
- agent_logs - System logging
- agent_deployments - Deployment tracking

Security Features

- Row Level Security (RLS) - Tenant isolation
- UUID Primary Keys - Secure identifiers
- Timestamp Tracking - Audit trail
- Indexed Queries - Performance optimization
- Views & Functions - Data abstraction

API Endpoints

Agent Factory

- POST /api/agent-factory
- GET /api/agent-factory
- PUT /api/agent-factory/{id}
- DELETE /api/agent-factory/{id}

Sentiment Agent

- POST /api/sentiment-agent
- GET /api/sentiment-agent/health
- GET /api/sentiment-agent/status

Recommendation Agent

- POST /api/recommendation-agent
- GET /api/recommendation-agent/health
- GET /api/recommendation-agent/status

Performance Monitor

- POST /api/performance-monitor
- GET /api/performance-monitor/health
- GET /api/performance-monitor/report

Panduan Deployment

1 Setup Supabase

Buat project baru di Supabase dan setup database schema:

```
# 1. Buat project di https://supabase.com
# 2. Copy URL dan Anon Key
# 3. Jalankan schema.sql di SQL Editor
# 4. Enable Row Level Security
# 5. Setup Authentication
```

2 Setup GitHub Repository

Buat repository dan upload semua file implementasi:

```
git init
git add .
git commit -m "Initial FMAA implementation"
git branch -M main
git remote add origin https://github.com/username/fmaa-ecosystem.git
git push -u origin main
```

3 Deploy ke Vercel

Deploy aplikasi ke Vercel dan setup environment variables:

```
# 1. Connect GitHub repository ke Vercel
# 2. Set Environment Variables:
SUPABASE_URL=your_supabase_url
SUPABASE_ANON_KEY=your_anon_key
HUGGINGFACE_API_KEY=your_hf_key
# 3. Deploy otomatis dari GitHub
```

4 Setup Hugging Face

Dapatkan API key untuk mengakses model AI:

```
# 1. Buat akun di https://huggingface.co
# 2. Generate API Token di Settings
# 3. Set token di Vercel Environment Variables
# 4. Test API access
```

5 Build Mobile App

Compile dan install Flutter application:

```
cd mobile-app
flutter pub get
flutter build apk --release
# Install APK ke Android device
```

Konfigurasi Environment

Vercel Environment Variables

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your-anon-key
SUPABASE_SERVICE_ROLE_KEY=your-service-key
HUGGINGFACE_API_KEY=your-hf-token
NODE_ENV=production
```

Flutter Configuration

```
# lib/config/constants.dart
const String SUPABASE_URL = 'your-url';
const String SUPABASE_ANON_KEY = 'your-key';
const String API_BASE_URL = 'https://your-vercel-app.vercel.app';
```

Panduan Penggunaan

Menggunakan Mobile Dashboard

- Buka aplikasi FMAA Dashboard di Android
- Login dengan akun Supabase
- Navigasi ke tab "Agents" untuk melihat semua agents
- Klik agent untuk melihat detail dan metrics
- Gunakan tab "Tasks" untuk memonitor task yang sedang berjalan
- Dashboard menampilkan real-time status sistem

Menggunakan API

```
// Create new agent
const response = await fetch('/api/agent-factory', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    agentType: 'sentiment-analysis',
    config: { model: 'roberta-base' }
  })
});

// Analyze sentiment
const sentiment = await fetch('/api/sentiment-agent', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    text: 'I love this product!',
    taskId: 'uuid-task-id'
  })
});
```

Monitoring & Maintenance

Health Checks

- Agent status monitoring
- API endpoint health
- Database connectivity
- Real-time performance metrics

Database Monitoring

- Query performance
- Connection pooling
- Storage usage
- Backup status

Maintenance Tasks

- Log cleanup
- Performance optimization
- Security updates
- Agent redeployment

Langkah Selanjutnya

Fase Berikutnya

- Implementasi Multi-tenancy penuh
- Sistem billing dan subscription
- Advanced analytics dashboard
- Auto-scaling dan load balancing

Optimisasi

- Caching layer untuk performance
- Rate limiting dan security
- Automated testing pipeline
- Container orchestration

Support & Resources



Documentation

Dokumentasi lengkap API dan implementation guide



Source Code

Repository lengkap dengan semua implementasi



Support

Community support dan troubleshooting

FMAA Ecosystem

Federated Micro-Agents Architecture - Complete Implementation

Production Ready Mobile Optimized Cloud Native

Made with Genspark